

实验七：I/O 模型演进：从 C10K 到协程（学生版）

1. 实验目标

1. 观察多线程阻塞 I/O 模型在线程资源受限时的容量瓶颈。
2. 理解阻塞与非阻塞、同步与异步是两组不同概念。
3. 通过相同负载比较忙轮询与 `epoll` 的 CPU 时间差异。
4. 掌握 `epoll_create1`、`epoll_ctl`、`epoll_wait` 的基本使用方法。
5. 可选地将实验三的协程与 `epoll` 结合，理解协程如何保存执行位置并改善多轮业务代码的组织方式。
6. 通过自选进阶任务完成一次明显超出基础实验规模的高并发服务器扩展。

2. 问题背景：C10K

C10K 问题关注一台服务器如何同时管理约一万个网络连接。大量连接通常不会持续传输数据，而是长时间等待请求或下一轮消息。因此，服务器不仅要能够建立连接，还要以较低的线程、内存和 CPU 开销等待连接变为可读或可写。

本实验不要求恰好建立 10000 个连接，而是通过统一的慢连接负载观察相同的资源增长规律：

多线程阻塞 I/O

→ 线程数量和栈空间随连接数增长

单线程非阻塞忙轮询

→ 避免为每个连接创建线程

→ CPU 消耗在反复查询尚未就绪的连接

`epoll`

→ 由内核报告就绪连接

→ 应用程序只处理已经就绪的连接

`epoll` + 协程（可选）

→ 保留高效的事件等待

→ 将多轮业务恢复为顺序代码

实验统一使用 4000 个慢连接，是为了在不同实验环境中稳定复现上述现象。C10K 表示高并发连接带来的设计问题和资源挑战，不是必须达到的固定验收数字。

3. 实验环境与编译要求

本实验要求原生 Linux 或能够完整提供 Linux 系统调用的实验环境。

可选任务四还会使用：

- `ucontext` 协程代码；
- `nc` 交互式 TCP 客户端，用于两轮登录测试。

选择完成任务四且执行 `nc -h` 提示命令不存在时，在 Ubuntu、Debian 或 WSL Ubuntu 中安装：

```
sudo apt update
sudo apt install netcat-openbsd
```

安装后使用的命令仍为 `nc`。如果实验环境没有软件安装权限，也可以使用系统已有的 `ncat`，对应测试命令为：

```
ncat 127.0.0.1 8080
```

本手册提供统一测试客户端、三个基础必做任务框架和一个可选任务框架。资源限制、计时、日志等辅助代码已经给出。学生需要根据 TODO 的功能要求完成连接管理、错误处理和控制流。

阶段 1、2、3 使用相同实验条件：

- 服务端地址空间上限：512MB；
- 服务端文件描述符上限：最多提升到 10000；
- 客户端建立 4000 个连接并保持 10 秒；
- 三个阶段处理相同的固定 HTTP 请求。

如果当前 Linux 环境的文件描述符 hard limit 或临时端口范围不足，应记录实际可建立的连接数，并在三个阶段使用同一连接数。

4. 统一测试工具：慢连接客户端

普通 HTTP 压测工具会在建立连接后立即发送请求，服务端连接可能很快结束，难以观察大量等待连接。

本实验使用 `hold_client.c`：

```
建立大量 TCP 连接
→ 保持指定时间，不发送请求
→ 向所有连接发送 HTTP 请求
→ 读取并统计服务端响应
```

编译命令：

```
gcc -Wall -Wextra -O2 hold_client.c -o hold_client
```

客户端最终统计：

- 有效 HTTP 响应；
- 无响应关闭；
- 读取超时；
- 连接重置或无效响应。

阶段 1、2、3 必须使用相同的连接数和保持时间。

完整代码如下，将其保存为 `hold_client.c`：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
```

```

#include <signal.h>
#include <errno.h>
#include <sys/resource.h>
#include <sys/socket.h>
#include <sys/time.h>
#include <arpa/inet.h>

static const char *REQUEST =
    "GET / HTTP/1.1\r\n"
    "Host: 127.0.0.1\r\n"
    "Connection: close\r\n"
    "\r\n";

static void raise_fd_limit(int count)
{
    struct rlimit limit;
    if (getrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("getrlimit");
        exit(1);
    }

    rlim_t target = (rlim_t)count + 100;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;

    limit.rlim_cur = target;
    if (setrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("setrlimit");
        exit(1);
    }
}

int main(int argc, char *argv[])
{
    if (argc != 3) {
        fprintf(stderr, "用法: %s <连接数> <保持秒数>\n", argv[0]);
        return 1;
    }

    int target = atoi(argv[1]);
    int hold_seconds = atoi(argv[2]);
    if (target <= 0 || hold_seconds < 0) {
        fprintf(stderr, "连接数必须大于 0, 保持时间不能为负数\n");
        return 1;
    }

    raise_fd_limit(target);
    signal(SIGPIPE, SIG_IGN);

    int *fds = malloc((size_t)target * sizeof(int));
    if (fds == NULL) {
        perror("malloc");
        return 1;
    }

```

```

}

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) != 1) {
    fprintf(stderr, "inet_pton failed\n");
    free(fds);
    return 1;
}

int connected = 0;
for (int i = 0; i < target; i++) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) {
        perror("socket");
        break;
    }

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect");
        close(fd);
        break;
    }

    struct timeval timeout = {
        .tv_sec = 5,
        .tv_usec = 0,
    };
    if (setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO,
        &timeout, sizeof(timeout)) < 0) {
        perror("setsockopt SO_RCVTIMEO");
        close(fd);
        break;
    }

    fds[connected++] = fd;
    if (connected % 500 == 0)
        printf("已建立 %d 个连接\n", connected);
}

printf("共建立 %d 个连接, 保持 %d 秒且暂不发送数据\n",
    connected, hold_seconds);
sleep((unsigned int)hold_seconds);

int sent = 0;
for (int i = 0; i < connected; i++) {
    ssize_t n = write(fds[i], REQUEST, strlen(REQUEST));
    if (n > 0)
        sent++;
}

```

```

printf("write() 成功返回的连接: %d\n", sent);

int response_count = 0;
int closed_count = 0;
int timeout_count = 0;
int error_count = 0;

for (int i = 0; i < connected; i++) {
    char response[512];
    ssize_t n = read(fds[i], response, sizeof(response) - 1);

    if (n > 0) {
        response[n] = '\0';
        if (strstr(response, "HTTP/1.1 200 OK") != NULL)
            response_count++;
        else
            error_count++;
    } else if (n == 0) {
        /* TCP 已关闭, 但服务端没有返回任何 HTTP 数据。 */
        closed_count++;
    } else if (errno == EAGAIN || errno == EWOULDBLOCK) {
        /* SO_RCVTIMEO 到期, 仍未收到响应。 */
        timeout_count++;
    } else {
        /* 例如 ECONNRESET: 服务端接收后立即关闭连接。 */
        error_count++;
    }
}

printf("有效 HTTP 响应: %d | 无响应关闭: %d | "
       "读取超时: %d | 重置或无效响应: %d\n",
       response_count,
       closed_count,
       timeout_count,
       error_count);

for (int i = 0; i < connected; i++)
    close(fds[i]);

free(fds);
return 0;
}

```

5. 基础实验任务

三个基础必做任务形成 I/O 模型演进主线, 任务四用于可选体验协程对代码结构的改善:

阶段 1: 多线程阻塞 I/O

→ 线程资源随连接数增长

阶段 2: 单线程 + 非阻塞 I/O

→ 解决线程爆炸

→ 忙轮询占用 CPU

阶段 3: `epoll` 短连接服务器

→ 解决忙轮询

过渡分析: `Reactor` 多轮业务

→ 显式状态和跨事件数据使代码复杂化

可选任务 4: `epoll` + 协程

→ 保留 `epoll` 的等待效率

→ 恢复顺序业务代码

5.1 任务一：同步阻塞 I/O + 多线程

完成 `stage1_threaded.c` 并运行程序，观察：

1. 服务端成功接收的连接数量；
2. 服务端首次出现 `pthread_create failed` 时的连接数量和错误信息；
3. 客户端成功建立的 TCP 连接数；
4. 客户端 `write()` 成功返回的连接数；
5. 客户端收到的有效 HTTP 响应数、无响应关闭数、读取超时数和错误数；
6. 服务端每接收 500 个连接时输出的 `Threads`、`VmSize` 和 `VmRSS`。

完整框架如下，将其保存为 `stage1_threaded.c`：

```
/*
 * 阶段 1: 阻塞 I/O + 多线程
 *
 * 编译: gcc -Wall -o stage1 stage1_threaded.c -lpthread
 * 运行: ./stage1
 * 测试: ./hold_client 4000 10
 *
 * 实验条件: 512MB 地址空间，每个工作线程使用 256KB 栈。
 * 预期结果: 并发连接持续增加后，pthread_create 最终返回 EAGAIN。
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/resource.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <errno.h>

#define PORT 8080
#define BACKLOG 4096
#define BUF_SIZE 4096
#define ADDRESS_SPACE_MB 512
```

```

#define THREAD_STACK_KB    256
#define FILE_LIMIT          10000

static const char *HTTP_RESP =
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/plain\r\n"
    "Content-Length: 13\r\n"
    "Connection: close\r\n"
    "\r\n"
    "Hello, world!";

/*
 * 实验辅助函数：限制当前服务端进程的资源。
 *
 * 这里不是服务器业务逻辑的一部分，而是为了让不同机器上都能较稳定地
 * 观察到“线程数量增长后 pthread_create 失败”的现象。
 *
 * 只限制当前服务端进程的虚拟地址空间，不会限制整个 Linux 或宿主机。
 * 保留原有 hard limit，仅降低 soft limit，普通用户即可执行。
 */
static void limit_process_resources(void)
{
    struct rlimit limit;
    if (getrlimit(RLIMIT_AS, &limit) < 0) {
        perror("getrlimit RLIMIT_AS");
        exit(1);
    }

    rlim_t target = (rlim_t)ADDRESS_SPACE_MB * 1024 * 1024;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;

    limit.rlim_cur = target;
    if (setrlimit(RLIMIT_AS, &limit) < 0) {
        perror("setrlimit RLIMIT_AS");
        exit(1);
    }

    /*
     * 提高当前进程的 fd soft limit，避免先在约 1024 个连接处
     * 因 accept 返回 EMFILE 而看不到线程资源瓶颈。
     */
    if (getrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("getrlimit RLIMIT_NOFILE");
        exit(1);
    }

    target = FILE_LIMIT;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;

    limit.rlim_cur = target;
    if (setrlimit(RLIMIT_NOFILE, &limit) < 0) {

```

```

        perror("setrlimit RLIMIT_NOFILE");
        exit(1);
    }
}

/*
 * 实验辅助函数：输出当前服务端进程的线程数和内存信息。
 *
 * /proc/self/status 是 Linux 提供的进程状态文件。
 * Threads 用来观察线程数量是否随连接数增长；
 * VmSize 用来观察虚拟地址空间是否接近 512MB 限制；
 * VmRSS 用来观察实际驻留物理内存。
 */
static void print_process_status(long long conn_count)
{
    FILE *fp = fopen("/proc/self/status", "r");
    if (fp == NULL) {
        perror("fopen /proc/self/status");
        return;
    }

    char line[256];
    char threads[256] = "";
    char vm_size[256] = "";
    char vm_rss[256] = "";

    while (fgets(line, sizeof(line), fp) != NULL) {
        if (strncmp(line, "Threads:", 8) == 0)
            snprintf(threads, sizeof(threads), "%s", line);
        else if (strncmp(line, "VmSize:", 7) == 0)
            snprintf(vm_size, sizeof(vm_size), "%s", line);
        else if (strncmp(line, "VmRSS:", 6) == 0)
            snprintf(vm_rss, sizeof(vm_rss), "%s", line);
    }

    fclose(fp);

    printf("[Stage 1] 资源状态（已接收 %lld 个连接）\n", conn_count);
    printf("  %s", threads);
    printf("  %s", vm_size);
    printf("  %s", vm_rss);
    fflush(stdout);
}

static void *handle_client(void *arg)
{
    /*
     * TODO 1: 完成一个阻塞式客户端处理函数。
     *
     * 要求：
     * - main 传入的 arg 是 int *, 其中保存一个 client_fd;
     * - 先把 fd 取出来, 再释放 arg, 避免线程结束后泄漏内存;
     * - 阻塞读取一次客户端请求;
    */

```



```

    * - 只有确实读到数据时才发送 HTTP_RESP;
    * - 无论读写结果如何，最后都要关闭客户端 fd 并结束线程函数。
    */
/* TODO: 从 arg 中取出 client_fd，并释放 main 中 malloc 的参数。 */

/* TODO: 定义接收缓冲区，完成一次阻塞读取。 */

/* TODO: 如果读到了数据，发送 HTTP_RESP。 */

/* TODO: 关闭 client_fd，并返回 NULL。 */
return NULL;
}

int main(void)
{
    limit_process_resources();

    /*
     * TODO 2: 创建并初始化监听 socket。
     *
     * 需要依次完成:
     * 1. 创建 IPv4 TCP 监听 fd;
     * 2. setsockopt(SO_REUSEADDR) 已给出;
     * 3. bind() 绑定到 addr;
     * 4. listen() 进入监听状态。
     *
     * socket、bind、listen 任一步失败都应打印错误并退出。
     */
    int listen_fd;
    /* TODO: 创建 IPv4 TCP socket，并赋值给 listen_fd。 */
    if (listen_fd < 0) {
        perror("socket");
        exit(1);
    }

    int opt = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = {
        .sin_family    = AF_INET,
        .sin_port      = htons(PORT),
        .sin_addr.s_addr = INADDR_ANY,
    };

    /* TODO: 把 listen_fd 绑定到 addr。 */

    /* TODO: 让 listen_fd 进入监听状态，等待队列长度使用 BACKLOG。 */

    /*
     * 统一线程属性：线程创建后即处于 detached 状态，并使用固定栈大小。
     * 256KB 远小于常见的默认线程栈，但足够本实验的简单处理函数使用。
     */
    pthread_attr_t attr;

```

```

int ret = pthread_attr_init(&attr);
if (ret != 0) {
    fprintf(stderr, "pthread_attr_init: %s\n", strerror(ret));
    exit(1);
}

ret = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
if (ret != 0) {
    fprintf(stderr, "pthread_attr_setdetachstate: %s\n", strerror(ret));
    exit(1);
}

ret = pthread_attr_setstacksize(
    &attr, (size_t)THREAD_STACK_KB * 1024);
if (ret != 0) {
    fprintf(stderr, "pthread_attr_setstacksize: %s\n", strerror(ret));
    exit(1);
}

/* 实验日志：提示服务端已启动，并打印本阶段的关键资源设置。 */
printf("[Stage 1] 服务器启动，监听 :%d\n", PORT);
printf("[Stage 1] 地址空间限制：%d MB，每线程栈：%d KB\n",
    ADDRESS_SPACE_MB, THREAD_STACK_KB);
printf("[Stage 1] 测试命令：./hold_client 4000 10\n\n");

long long conn_count = 0;
int first_thread_failure_reported = 0;

while (1) {
    int *client_fd = malloc(sizeof(int));
    if (client_fd == NULL) {
        perror("malloc client_fd");
        continue;
    }

    /*
     * TODO 3：完成“接收连接并创建工作线程”的完整流程。
     *
     * client_fd 是 main 分配的一块内存，用来把 fd 交给新线程。
     * 线程创建成功后，这块内存由 handle_client 释放。
     * 线程创建失败时，main 必须自己关闭 fd 并释放 client_fd。
     *
     * 线程必须使用 attr；创建失败时服务器应继续接收后续连接。
     */
    /* TODO：接收一个新连接，并把返回的客户端 fd 保存到 *client_fd。 */
    if (*client_fd < 0) {
        perror("accept");
        free(client_fd);
        continue;
    }

    conn_count++;

```

```

pthread_t tid;
/*
 * TODO: 创建工作线程，线程属性使用 attr，
 * 线程入口使用 handle_client，参数使用 client_fd，
 * 并把线程创建结果保存到 ret。
 *
 * 注意: pthread_create 成功后，不要在 main 中 free(client_fd)，
 * 否则工作线程可能拿到已经释放的参数。
 */
if (ret != 0) {
    if (!first_thread_failure_reported) {
        fprintf(stderr,
            "[Stage 1] pthread_create first failed after "
            "%lld connections: %s\n",
            conn_count, strerror(ret));
        print_process_status(conn_count);
        first_thread_failure_reported = 1;
    }
    /* TODO: 关闭未交给工作线程的 *client_fd，并释放 client_fd。 */
    continue;
}

if (conn_count % 500 == 0) {
    /* 实验日志：每 500 个连接输出一次资源状态，便于观察增长趋势。 */
    printf("[Stage 1] 已接受 %lld 个连接\n", conn_count);
    print_process_status(conn_count);
}
}

pthread_attr_destroy(&attr);
close(listen_fd);
return 0;
}

```

编译运行：

```

gcc -Wall -Wextra -O2 stage1_threaded.c -o stage1 -lpthread
./stage1

```

另开终端：

```

./hold_client 4000 10

```

预期现象：

- 连接保持期间，大量线程阻塞在 `read()`；
- 线程数量随连接数近似线性增长；
- 达到线程、PID 或地址空间限制后，`pthread_create()` 返回 `EAGAIN`；
- 服务端不会故意崩溃，而是关闭无法分配工作线程的新连接；
- 客户端的有效 HTTP 响应数可能少于成功建立的 TCP 连接数。

任务一只要求提交：

- 完整的 `stage1_threaded.c`；
- 最终运行结果。

5.2 任务二：同步非阻塞 I/O + 忙轮询

本阶段提供 `stage2_busypoll.c` 文件级框架。辅助代码已经完成，学生负责非阻塞设置、监听 socket 初始化和整个忙轮询连接管理流程。

所有 socket 都设置为非阻塞，服务端只使用一个线程管理连接数组：

```
while (1) {
    /* 尝试 accept 新连接。 */

    for (int i = 0; i < client_count; i++) {
        /* 对每个连接尝试 read。 */
    }
}
```

连接没有数据时：

```
read(fd, buf, size) == -1
errno == EAGAIN
```

代码不会被阻塞，但会立即进入下一次循环。

完整框架如下，将其保存为 `stage2_busypoll.c`：

```
/*
 * 阶段 2: CPU 燃烧器（非阻塞 I/O + 忙轮询 Busy Polling）
 *
 * 编译: gcc -Wall -o stage2 stage2_busypoll.c
 * 运行: ./stage2
 * 测试: ./hold_client 4000 10
 *
 * 痛点: CPU 占用率飙升到 100%，哪怕没有任何请求
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <time.h>
#include <sys/resource.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT      8080
```

```

#define BACKLOG      1024
#define MAX_CLIENTS 10240
#define BUF_SIZE     4096
#define ADDRESS_SPACE_MB 512
#define FILE_LIMIT    10000
#define TEST_CONNECTIONS 4000

static const char *HTTP_RESP =
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/plain\r\n"
    "Content-Length: 13\r\n"
    "Connection: close\r\n"
    "\r\n"
    "Hello, world!";

static struct timespec wall_start;
static struct timespec cpu_start;
static int measuring = 0;

/*
 * 实验辅助函数：计算两个时间点之间相差多少秒。
 * CLOCK_MONOTONIC 用来统计真实经过时间，CLOCK_PROCESS_CPUTIME_ID
 * 用来统计当前进程真正消耗的 CPU 时间。
 */
static double elapsed_seconds(struct timespec start, struct timespec end)
{
    return (double)(end.tv_sec - start.tv_sec)
        + (double)(end.tv_nsec - start.tv_nsec) / 1000000000.0;
}

/*
 * 实验辅助函数：开始记录“慢连接保持阶段”的时间。
 * 当连接数达到 TEST_CONNECTIONS 后调用，用来比较任务二和任务三
 * 在同样等待时间内消耗了多少 CPU。
 */
static void start_measurement(void)
{
    clock_gettime(CLOCK_MONOTONIC, &wall_start);
    clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &cpu_start);
    measuring = 1;
}

/*
 * 实验辅助函数：结束计时并打印结果。
 * 实际时间接近客户端保持连接的时间；CPU 时间越大，说明服务器
 * 在等待期间越忙。忙轮询会让 CPU 时间接近实际时间。
 */
static void finish_measurement(void)
{
    struct timespec wall_end;
    struct timespec cpu_end;

    clock_gettime(CLOCK_MONOTONIC, &wall_end);

```

```

clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &cpu_end);

printf("[Stage 2] 保持阶段: 实际时间 %.2f 秒 | CPU 时间 %.2f 秒\n",
       elapsed_seconds(wall_start, wall_end),
       elapsed_seconds(cpu_start, cpu_end));
measuring = 0;
}

/*
 * 实验辅助函数: 设置当前进程资源上限。
 * 任务二不需要创建大量线程, 但仍沿用相同资源条件, 保证和任务一、
 * 任务三的测试环境一致, 同时提高 fd 上限以容纳大量慢连接。
 */
static void limit_process_resources(void)
{
    struct rlimit limit;

    if (getrlimit(RLIMIT_AS, &limit) < 0) {
        perror("getrlimit RLIMIT_AS");
        exit(1);
    }

    rlim_t target = (rlim_t)ADDRESS_SPACE_MB * 1024 * 1024;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;
    limit.rlim_cur = target;

    if (setrlimit(RLIMIT_AS, &limit) < 0) {
        perror("setrlimit RLIMIT_AS");
        exit(1);
    }

    if (getrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("getrlimit RLIMIT_NOFILE");
        exit(1);
    }

    target = FILE_LIMIT;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;
    limit.rlim_cur = target;

    if (setrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("setrlimit RLIMIT_NOFILE");
        exit(1);
    }
}

/* 将 fd 设置为非阻塞模式 */
static void set_nonblocking(int fd)
{
    /*
     * TODO 1: 在保留 fd 原有状态标志的前提下启用非阻塞模式。

```

```

*
* 要求：
* - 先取得 fd 原有状态标志；
* - 在原有标志基础上加入 O_NONBLOCK；
* - 两次 fcntl 操作失败时都要打印对应错误并退出；
* - 不要直接设置成 O_NONBLOCK，否则可能覆盖 fd 原有标志。
*/
int flags;
/* TODO: 读取 fd 当前的状态标志并保存到 flags。 */
if (flags < 0) { perror("fcntl F_GETFL"); exit(1); }
/* TODO: 在 flags 基础上加入 O_NONBLOCK；失败时退出。 */
}

int main(void)
{
    limit_process_resources();

    /*
    * TODO 2: 创建监听 socket，完成复用地址、绑定、监听，
    * 并将监听 socket 设置为非阻塞。
    *
    * 需要依次完成：
    * 1. 创建 IPv4 TCP 监听 fd；
    * 2. setsockopt(SO_REUSEADDR) 已给出；
    * 3. bind() 绑定到 addr；
    * 4. listen() 使用 BACKLOG 进入监听状态；
    * 5. 调用 set_nonblocking(listen_fd)。
    *
    * socket、bind、listen 任一步失败都应打印错误并退出。
    */
    int listen_fd;
    /* TODO: 创建 IPv4 TCP socket，并赋值给 listen_fd。 */
    if (listen_fd < 0) { perror("socket"); exit(1); }

    int opt = 1;
    setsockopt(listen_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));

    struct sockaddr_in addr = {
        .sin_family    = AF_INET,
        .sin_port      = htons(PORT),
        .sin_addr.s_addr = INADDR_ANY,
    };
    /* TODO: 把 listen_fd 绑定到 addr。 */

    /* TODO: 让 listen_fd 进入监听状态，等待队列长度使用 BACKLOG。 */

    /* TODO: 将 listen_fd 设置为非阻塞。 */

    /* 实验日志：提示服务端已启动，以及本阶段应使用的测试命令。 */
    printf("[Stage 2] 服务器启动，监听 :%d\n", PORT);
    printf("[Stage 2] 测试命令：./hold_client 4000 10\n\n");

    /* 保存所有客户端 fd 的数组 */

```

```

int clients[MAX_CLIENTS];
int client_count = 0;
memset(clients, -1, sizeof(clients));

char buf[BUF_SIZE];
/*
 * TODO 3: 实现完整的忙轮询主循环。
 *
 * 约束:
 * - 不得使用 select、poll、epoll、sleep 或工作线程;
 * - 每轮先接收当前能够取得的全部新连接;
 * - 使用 clients 数组保存活动连接;
 * - 对所有活动连接逐个尝试 read;
 * - EAGAIN/EWOULDBLOCK 时保留连接;
 * - 请求到达后回复并关闭;
 * - 对端关闭或发生永久错误时移除连接;
 * - 数组满时立即关闭新连接;
 * - 第 TEST_CONNECTIONS 个连接建立时调用 start_measurement;
 * - 第一条请求到达时调用 finish_measurement。
 */
while (1) {
    /* 尝试接受新连接（非阻塞，若无连接立即返回 -1/EAGAIN） */
    while (1) {
        int fd;
        /* TODO: 尝试接收新连接，并把返回的客户端 fd 保存到 fd。 */
        if (fd < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK)
                break;
            if (errno == EINTR || errno == ECONNABORTED)
                continue;
            perror("accept");
            break;
        }
        set_nonblocking(fd);
        if (client_count < MAX_CLIENTS) {
            /*
             * TODO: 把 fd 放入 clients 数组，并更新 client_count。
             * 第 TEST_CONNECTIONS 个连接放入数组后，开始测量保持阶段。
             */
            if (client_count == TEST_CONNECTIONS && !measuring) {
                start_measurement();
                printf("[Stage 2] 已接收 %d 个连接，开始测量保持阶段\n",
                    TEST_CONNECTIONS);
            }
        } else {
            fprintf(stderr, "客户端数组已满! \n");
            close(fd);
        }
    }

    /* 遍历所有客户端 fd，逐个尝试读取 */
    for (int i = 0; i < client_count; i++) {
        if (clients[i] < 0) continue;
    }
}

```



```

ssize_t n;
/* TODO: 尝试读取 clients[i], 把结果保存到 n。 */

if (n > 0) {
    if (measuring)
        finish_measurement();

    /*
     * TODO: 对 clients[i] 发送 HTTP_RESP, 关闭该 fd,
     * 并从 clients 数组移除当前连接。
     *
     * 移除时要保证 clients 数组中剩余 fd 仍然连续,
     * 并且不要跳过被移动到当前位置的连接。
     */
} else if (n == 0) {
    /* 对端关闭 */
    close(clients[i]);
    clients[i] = clients[--client_count];
    i--;
} else {
    /* n < 0 */
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        /*
         * TODO: 当前连接暂时没有数据。
         * 不要关闭 fd, 不要修改 clients 数组,
         * 直接保留连接等待下一轮扫描。
         */
    } else {
        perror("read");
        close(clients[i]);
        clients[i] = clients[--client_count];
        i--;
    }
}

}

}

close(listen_fd);
return 0;
}

```

编译运行:

```
gcc -Wall -Wextra -O2 stage2_busypoll.c -o stage2
./stage2
```

另开终端:

```
./hold_client 4000 10
```

当服务端接收第 4000 个连接时开始测量；第一条请求到达时结束测量。程序会打印：

```
[Stage 2] 保持阶段：实际时间 ... 秒 | CPU 时间 ... 秒
```

预期结果：

- 实际时间接近客户端设置的 10 秒；
- CPU 时间也接近 10 秒；
- 服务端只有一个执行线程；
- 任务二解决了任务一的线程爆炸，但把 CPU 浪费在重复调用 `accept()` 和 `read()` 上。

任务二只要求提交：

- 完整的 `stage2_busypoll.c`；
- CPU 时间测量结果；
- 与任务一线程数量的对比分析。

5.3 任务三：epoll + Reactor

任务三完成与任务二相同的短连接 HTTP 业务，但不再扫描全部客户端。

核心流程：

```
创建 epoll 实例
→ 注册监听 socket
→ epoll_wait 等待事件
→ 接收新连接并注册 EPOLLIN
→ 只处理 epoll 返回的就绪连接
```

基础任务使用水平触发 LT，不要求实现 ET。

将下面的完整文件级框架保存为 `stage3_epoll.c`。监听 socket 和非阻塞设置应复用自己在前两个任务中完成的代码；本阶段重点完成 epoll 初始化和 Reactor 事件循环。

```
/*
 * 阶段 3：事件驱动的救星（epoll + Reactor 模式）
 *
 * 编译：gcc -Wall -o stage3 stage3_epoll.c
 * 运行：./stage3
 * 测试：./hold_client 4000 10
 *
 * 等待方式：没有事件时线程阻塞在 epoll_wait()。
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <time.h>
```

```

#include <sys/resource.h>
#include <sys/socket.h>
#include <sys/epoll.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT      8080
#define BACKLOG   1024
#define MAX_EVENTS 1024
#define BUF_SIZE  4096
#define ADDRESS_SPACE_MB 512
#define FILE_LIMIT 10000
#define TEST_CONNECTIONS 4000

static const char *HTTP_RESP =
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/plain\r\n"
    "Content-Length: 13\r\n"
    "Connection: close\r\n"
    "\r\n"
    "Hello, world!";

static struct timespec wall_start;
static struct timespec cpu_start;
static int measuring = 0;

/*
 * 实验辅助函数：计算两个时间点之间相差多少秒。
 * 任务三使用它和任务二做同口径比较：真实经过时间看等待阶段持续多久，
 * 进程 CPU 时间看服务器在等待期间是否仍然忙碌。
 */
static double elapsed_seconds(struct timespec start, struct timespec end)
{
    return (double)(end.tv_sec - start.tv_sec)
        + (double)(end.tv_nsec - start.tv_nsec) / 1000000000.0;
}

/*
 * 实验辅助函数：开始记录慢连接保持阶段。
 * 第 TEST_CONNECTIONS 个连接接入后开始计时，此后客户端暂时不发送请求，
 * 用来观察 epoll_wait 是否让服务器线程休眠。
 */
static void start_measurement(void)
{
    clock_gettime(CLOCK_MONOTONIC, &wall_start);
    clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &cpu_start);
    measuring = 1;
}

/*
 * 实验辅助函数：结束计时并打印实际时间和 CPU 时间。
 * 如果 epoll 使用正确，实际时间会接近保持时间，而 CPU 时间应明显小于
 * 忙轮询版本。

```

```

*/
static void finish_measurement(void)
{
    struct timespec wall_end;
    struct timespec cpu_end;

    clock_gettime(CLOCK_MONOTONIC, &wall_end);
    clock_gettime(CLOCK_PROCESS_CPUTIME_ID, &cpu_end);

    printf("[Stage 3] 保持阶段: 实际时间 %.2f 秒 | CPU 时间 %.2f 秒\n",
        elapsed_seconds(wall_start, wall_end),
        elapsed_seconds(cpu_start, cpu_end));
    measuring = 0;
}

/*
 * 实验辅助函数: 设置当前进程资源上限。
 * 任务三沿用任务二的连接规模和资源条件, 便于只比较“等待方式”
 * 从忙轮询变成 epoll 后的 CPU 时间差异。
 */
static void limit_process_resources(void)
{
    struct rlimit limit;

    if (getrlimit(RLIMIT_AS, &limit) < 0) {
        perror("getrlimit RLIMIT_AS");
        exit(1);
    }

    rlim_t target = (rlim_t)ADDRESS_SPACE_MB * 1024 * 1024;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;
    limit.rlim_cur = target;

    if (setrlimit(RLIMIT_AS, &limit) < 0) {
        perror("setrlimit RLIMIT_AS");
        exit(1);
    }

    if (getrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("getrlimit RLIMIT_NOFILE");
        exit(1);
    }

    target = FILE_LIMIT;
    if (limit.rlim_max != RLIM_INFINITY && target > limit.rlim_max)
        target = limit.rlim_max;
    limit.rlim_cur = target;

    if (setrlimit(RLIMIT_NOFILE, &limit) < 0) {
        perror("setrlimit RLIMIT_NOFILE");
        exit(1);
    }
}

```

```

}

/* 将 fd 设置为非阻塞模式 */
static void set_nonblocking(int fd)
{
    /*
     * TODO: 复用任务二的实现，在保留原状态标志的基础上
     * 将 fd 设置为非阻塞。
     *
     * 提示：
     * - 先读取 fd 当前状态标志；
     * - 再在原标志基础上加入 O_NONBLOCK；
     * - 任一步失败都打印错误并退出。
     */
}

int main(void)
{
    limit_process_resources();

    /*
     * TODO: 复用任务二的监听 socket 初始化代码。
     *
     * 要求：
     * - 创建 IPv4 TCP socket；
     * - 设置地址复用；
     * - 绑定 PORT 并开始监听；
     * - 将监听 fd 设置为非阻塞；
     * - 把最终的监听 fd 保存到 listen_fd。
     *
     * 可以直接复用任务二中已经完成的监听 socket 初始化顺序。
     */
    int listen_fd;

    /*
     * TODO 1: 创建 epoll 实例并注册 listen_fd 的可读事件。
     * 注册失败时应释放资源并退出。
     */
    int epoll_fd;
    /* TODO: 创建 epoll 实例，并将结果保存到 epoll_fd。 */
    if (epoll_fd < 0) { perror("epoll_create1"); exit(1); }

    struct epoll_event ev;
    memset(&ev, 0, sizeof(ev));
    /*
     * TODO: 把 listen_fd 作为可读事件加入 epoll 监听集合。
     *
     * 要求：
     * - 监听 fd 只需要关注可读事件；
     * - 事件中要能区分返回的是 listen_fd；
     * - 注册失败时打印错误并退出。
     */
}

```

```

/* 实验日志：提示服务端已启动，并强调空闲时应阻塞在 epoll_wait。 */
printf("[Stage 3] epoll 服务器启动，监听 :%d\n", PORT);
printf("[Stage 3] 测试命令： ./hold_client 4000 10\n");
printf("[Stage 3] 没有请求时线程阻塞在 epoll_wait, CPU 接近 0\n\n");

struct epoll_event events[MAX_EVENTS];
char buf[BUF_SIZE];
long long conn_count = 0;

/*
 * TODO 2: 实现完整 Reactor 事件循环。
 *
 * 约束：
 * - epoll_wait 在没有事件时必须阻塞；
 * - 监听 fd 就绪时持续 accept，直到 EAGAIN；
 * - 新客户端必须设置为非阻塞并注册 EPOLLIN；
 * - 只处理 events 数组返回的就绪 fd，不得扫描全部连接；
 * - 如果事件包含 EPOLLERR 或 EPOLLHUP，应删除并关闭该 fd；
 * - 正确区分数据到达、对端关闭、EAGAIN 和永久错误；
 * - 关闭连接前从 epoll 删除；
 * - 保持与任务二相同的测量起止条件。
 */
while (1) {
    int n;
    /*
     * TODO: 等待 epoll 事件，并把就绪事件数量保存到 n。
     *
     * 要求：没有事件时阻塞等待；返回后只处理 events
     * 数组中实际就绪的 n 个事件。
     */
    if (n < 0) {
        if (errno == EINTR) continue;
        perror("epoll_wait");
        break;
    }

    for (int i = 0; i < n; i++) {
        int fd = events[i].data.fd;

        if (fd == listen_fd) {
            /*
             * 监听 fd 就绪：说明有新连接。
             * 因为 listen_fd 是非阻塞的，所以要循环 accept，
             * 直到 EAGAIN 表示暂时没有更多连接。
             */
            while (1) {
                int client_fd;
                /*
                 * TODO: 复用任务二的非阻塞 accept 和错误处理。
                 * 把新连接保存到 client_fd；没有更多连接时结束循环。
                 *
                 * 要求：
                 * - 从监听 fd 接收新连接；

```

```

        * - EAGAIN/EWOULDBLOCK 表示本轮没有更多连接，应 break;
        * - EINTR/ECONNABORTED 可以继续 accept;
        * - 其他错误打印 "accept" 并结束本轮 accept。
    */
    if (client_fd < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK)
            break;
        perror("accept");
        break;
    }

    set_nonblocking(client_fd);

    struct epoll_event client_ev;
    memset(&client_ev, 0, sizeof(client_ev));

    /*
     * TODO: 把 client_fd 作为可读事件加入 epoll 监听集合。
     *
     * 要求:
     * - 新客户端只需要关注可读事件;
     * - 事件中要能找回这个 client_fd;
     * - 注册失败时打印错误并 close(client_fd),
     *   不要把注册失败的 fd 留给事件循环。
    */

    conn_count++;
    if (conn_count == TEST_CONNECTIONS && !measuring) {
        start_measurement();
        printf("[Stage 3] 已接收 %d 个连接，开始测量保持阶段\n",
            TEST_CONNECTIONS);
    }

    if (conn_count % 1000 == 0)
        printf("[Stage 3] 已接受 %lld 个连接\n", conn_count);
}
} else {
    /*
     * TODO: 如果 events[i].events 包含 EPOLLERR 或 EPOLLHUP,
     * 说明连接已经出错或挂起，应从 epoll 删除 fd 并关闭，
     * 然后 continue 处理下一个事件。
    */

    ssize_t bytes;
    /*
     * TODO: 读取当前就绪的客户端 fd，并把返回值保存到 bytes。
     *
     * fd 是本次 epoll 返回的客户端 fd，不是 listen_fd。
    */

    if (bytes > 0) {
        if (measuring)
            finish_measurement();
    }
}

```

```

        /*
        * TODO: 复用前两个任务的响应与关闭逻辑，
        * 并在关闭前从 epoll 中删除 fd。
        *
        * 要求：
        * - 发送固定 HTTP 响应；
        * - 从 epoll 中删除该 fd；
        * - 关闭 fd 释放连接。
        */
    } else if (bytes == 0) {
        /*
        * TODO: 对端关闭连接。
        * 从 epoll 删除 fd，然后 close(fd)。
        */
    } else {
        /*
        * TODO: EAGAIN 时保留连接；其他错误输出信息，
        * 从 epoll 删除 fd 并关闭连接。
        *
        * 要求：
        * - errno 为 EAGAIN/EWOULDBLOCK 时什么也不做；
        * - 其他错误先 perror("read")；
        * - 再从 epoll 删除 fd 并关闭连接。
        */
    }
}

}

close(listen_fd);
close(epoll_fd);
return 0;
}

```

编译运行：

```
gcc -Wall -Wextra -O2 stage3_epoll.c -o stage3
./stage3
```

再运行：

```
./hold_client 4000 10
```

程序打印：

```
[Stage 3] 保持阶段：实际时间 ... 秒 | CPU 时间 ... 秒
```

预期结果：

- 实际时间接近 10 秒；
- CPU 时间接近 0 秒；
- 任务三仍然只有一个执行线程；
- `epoll_wait(..., -1)` 在没有事件时让线程休眠。

任务三要求提交：

- 完整的 `stage3_epoll.c`；
- 最终运行结果；
- 任务二、三 CPU 时间对比。

5.4 过渡：多轮业务下的 Reactor 复杂度

任务三只处理一次请求，因此事件处理逻辑比较简单。考虑下面的多轮登录协议：

```
读取一行用户名
→ 回复“请输入密码”
→ 等待一行密码
→ 回复“登录成功”
→ 关闭连接
```

Reactor 不能在处理用户名的事件分支中阻塞等待密码，否则单线程事件循环无法继续处理其他连接。因此程序通常需要显式定义连接状态：

```
typedef enum {
    WAIT_USERNAME,
    SEND_PASSWORD_PROMPT,
    WAIT_PASSWORD,
    SEND_LOGIN_RESULT
} LoginState;

typedef struct {
    int fd;
    LoginState state;
    char username[64];
    char recv_buf[256];
    size_t recv_used;
    char send_buf[256];
    size_t send_len;
    size_t send_pos;
} LoginConnection;
```

问题的根源在于：`epoll` 只会通知“某个 fd 现在可读或可写”，不会让业务函数自动从上次等待的位置继续执行。事件处理函数必须完成当前能够完成的工作，然后立即返回事件循环。下一次事件到达时，程序只能根据连接对象中保存的状态，判断应该从哪一步继续。

因此，原本从上到下的一段业务流程会被事件边界拆开：

读取用户名

- `read()` 返回 `EAGAIN`，保存接收进度并返回事件循环
- `fd` 再次可读，重新进入读事件处理函数
- 得到完整用户名，准备密码提示并改为关注可写事件
- `write()` 只发送一部分，保存发送位置并返回事件循环
- `fd` 再次可写，从原发送位置继续
- 提示发送完成，修改状态并重新关注可读事件
- 等待密码.....

每次事件到达时，代码都要同时回答下面几个问题：

1. 当前连接进行到了哪一个业务步骤；
2. 本次事件是可读、可写、错误还是超时；
3. 上一次接收或发送完成到了什么位置；
4. 本次处理结束后应关注 `EPOLLIN` 还是 `EPOLLOUT`；
5. 发生错误或对端关闭时，应释放哪些资源。

当业务只有“读取一次、回复一次”时，这些判断还比较集中。加入用户名、密码、验证码、Redis 查询、PostgreSQL 查询、磁盘读取和超时处理后，每一个新的等待点都可能增加新的状态、上下文字段和错误分支。例如，一个请求可能被拆成：

读取 HTTP 请求

- 等待 Redis 连接可写或可读
- 等待 PostgreSQL 连接可写或可读
- 等待磁盘读取完成
- 等待客户端连接可写

这里要区分两种常见写法。

一种写法是回调链。每一步完成后注册下一步处理函数，代码可能形成多层嵌套或多段彼此跳转的回调：

HTTP 读取完成回调

- Redis 查询完成回调
 - PostgreSQL 查询完成回调
 - 磁盘读取完成回调
 - 响应写回完成回调

这种组织方式容易发展成通常所说的“回调地狱”：正常流程、错误处理、超时和资源释放分散在多个回调中，阅读代码时需要沿着回调链跳转，才能还原原本从上到下的业务顺序。

另一种写法是手动状态机。程序不把下一步写成嵌套回调，而是用 `switch (state)` 统一分发：

```

switch (conn->state) {
case READING_HTTP:
    break;
case WAITING_REDIS:
    break;
case WAITING_PGSQL:
    break;
case WAITING_DISK:
    break;
case WRITING_RESPONSE:
    break;
}

```

这种写法可以避免源代码层面的回调嵌套，但代价是状态机和连接上下文会不断膨胀。原本可以放在函数调用栈和局部变量中的内容，例如 HTTP 解析进度、Redis 结果、PostgreSQL 结果、磁盘读取位置、响应写入位置、错误码和清理标记，都要显式放进连接对象中。它体现的主要问题不是“回调嵌套”，而是“状态外露”和“业务流程被事件边界拆散”。

本节为概念分析，无代码提交。完成前三个任务后，已经能够理解从阻塞 I/O、非阻塞忙轮询到 `epoll` 事件通知的主要演进过程。任务四改为可选体验，用协程完成同一个两轮登录协议，观察协程如何把分散的控制流恢复为顺序代码。

5.5 可选任务四：epoll + 协程

任务三和任务四在 I/O 等待机制上本质相同：底层都使用非阻塞 socket 和 `epoll` Reactor，由 `epoll_wait()` 等待 fd 就绪，再由应用程序调用 `read()` 或 `write()` 完成 I/O。任务四并不是一种比 `epoll` 更新的 I/O 模型，也不会改变 fd 就绪通知的含义。

两者的主要区别在代码编写方式。任务三由程序员显式维护连接状态、收发进度和下一步操作；如果再使用回调式异步接口，还可能出现回调链过深的问题。任务四由协程上下文保存执行位置和局部变量，使多轮业务可以按照“读取用户名、提示密码、读取密码、返回结果”的顺序编写。因此，任务四主要解决的是 Reactor 多阶段业务代码难写、难读和难维护的问题，而不是再次解决新的底层 I/O 性能问题。

前三个任务已经覆盖本实验要求掌握的核心 I/O 模型，因此本任务不作为基础必做内容。希望进一步理解“高效事件等待”和“顺序业务代码”如何结合的学生，可以完成本任务。

本任务使用协程实现两轮登录协议。网络初始化和 `epoll` 事件循环应复用自己在前置任务中完成的代码；实验三的协程创建、切换和恢复代码保持完整。本阶段重点实现“等待 fd 可读”的协程封装和顺序业务函数。

将下面框架保存为 `stage4_coroutine.c`：

```

#define _XOPEN_SOURCE 700

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <ucontext.h>
#include <sys/socket.h>

```

```

#include <sys/epoll.h>
#include <netinet/in.h>
#include <arpa/inet.h>

#define PORT            8080
#define BACKLOG         1024
#define MAX_EVENTS      1024
#define MAX_COROUTINES  4096
#define STACK_SIZE      (32 * 1024)
#define LISTEN_TAG      (1ULL << 63)

typedef enum {
    FREE,
    RUNNABLE,
    RUNNING,
    SUSPEND
} State;

typedef struct {
    ucontext_t ctx;
    char stack[STACK_SIZE];
    State state;
    void (*func)(void);
    int client_fd;
} Coroutine;

typedef struct {
    ucontext_t main_ctx;
    Coroutine coroutines[MAX_COROUTINES];
    int current_id;
} Scheduler;

static Scheduler scheduler;
static int epoll_fd;

static void set_nonblocking(int fd)
{
    /*
     * TODO: 复用任务二的实现，在保留原状态标志的基础上
     * 将 fd 设置为非阻塞。
     *
     * 提示：
     * - 先读取 fd 当前状态标志；
     * - 再在原标志基础上加入 O_NONBLOCK；
     * - 任一步失败都打印错误并退出。
     */
}

static void scheduler_init(void)
{
    scheduler.current_id = -1;

    for (int i = 0; i < MAX_COROUTINES; i++) {

```

```

        scheduler.coroutines[i].state = FREE;
        scheduler.coroutines[i].client_fd = -1;
    }
}

static int coroutine_get_current_id(void)
{
    return scheduler.current_id;
}

static void coroutine_yield(void)
{
    int id = scheduler.current_id;
    if (id < 0)
        return;

    Coroutine *co = &scheduler.coroutines[id];
    co->state = SUSPEND;
    scheduler.current_id = -1;
    swapcontext(&co->ctx, &scheduler.main_ctx);
}

static void coroutine_wrapper(void)
{
    int id = scheduler.current_id;
    Coroutine *co = &scheduler.coroutines[id];

    co->func();

    if (co->client_fd >= 0) {
        close(co->client_fd);
        co->client_fd = -1;
    }

    co->state = FREE;
    scheduler.current_id = -1;
    setcontext(&scheduler.main_ctx);
}

static int coroutine_spawn(void (*func)(void), int client_fd)
{
    int id = -1;

    for (int i = 0; i < MAX_COROUTINES; i++) {
        if (scheduler.coroutines[i].state == FREE) {
            id = i;
            break;
        }
    }

    if (id < 0)
        return -1;
}

```

```

Coroutine *co = &scheduler.coroutines[id];
co->state = RUNNABLE;
co->func = func;
co->client_fd = client_fd;

getcontext(&co->ctx);
co->ctx.uc_stack.ss_sp = co->stack;
co->ctx.uc_stack.ss_size = sizeof(co->stack);
co->ctx.uc_stack.ss_flags = 0;
co->ctx.uc_link = &scheduler.main_ctx;
makecontext(&co->ctx, coroutine_wrapper, 0);

return id;
}

static void coroutine_resume(int id)
{
    if (id < 0 || id >= MAX_COROUTINES)
        return;

    Coroutine *co = &scheduler.coroutines[id];
    if (co->state != RUNNABLE && co->state != SUSPEND)
        return;

    co->state = RUNNING;
    scheduler.current_id = id;
    swapcontext(&scheduler.main_ctx, &co->ctx);
}

/*
 * 协程版 read:
 * 没有数据时注册 EPOLLIN, 然后挂起当前协程。
 * fd 可读后, 调度器恢复该协程, 循环再次尝试 read。
 */
static ssize_t co_read(int fd, void *buf, size_t count)
{
    /*
     * TODO 1: 实现协程友好的读取。
     *
     * 行为要求:
     * - read 成功、对端关闭或发生永久错误时直接返回;
     * - EAGAIN/EWOULDBLOCK 时, 把 fd 的 EPOLLIN 事件与当前协程关联;
     * - 挂起当前协程, 让调度器继续服务其他连接;
     * - fd 就绪并恢复后, 清理本次事件注册并重新尝试 read;
     * - 正确处理重复注册和 epoll_ctl 失败。
     */
    for (;;) {
        ssize_t n = read(fd, buf, count);
        if (n >= 0)
            return n;

        if (errno != EAGAIN && errno != EWOULDBLOCK)
            return -1;
    }
}

```

```

int co_id;
/*
 * TODO: 取得当前协程 ID 并保存到 co_id。
 *
 * 提示:
 * - 当前正在执行的协程 ID 保存在 scheduler.current_id 中;
 * - 使用已有的调度器接口取得它;
 * - 如果 co_id < 0, 说明当前不在协程中, 应返回错误。
 */

struct epoll_event ev;
memset(&ev, 0, sizeof(ev));

/*
 * TODO: 把 fd 的 EPOLLIN 事件与 co_id 关联并注册到 epoll;
 * 挂起当前协程; 恢复后删除本次事件注册。
 *
 * 要求:
 * - 注册 fd 的可读事件;
 * - 事件数据中能找回当前协程;
 * - 注册成功后挂起当前协程;
 * - 协程恢复后删除本次等待;
 * - 如果 ADD 失败但 errno 是 EEXIST, 可以先 DEL 再 ADD;
 * - 其他 epoll_ctl 错误应返回 -1。
 */
}
}

/* TCP 没有消息边界, 因此按换行符读取一轮输入。 */
static ssize_t co_read_line(int fd, char *buf, size_t size)
{
    size_t used = 0;

    while (used + 1 < size) {
        char ch;
        ssize_t n = co_read(fd, &ch, 1);
        if (n <= 0)
            return n;

        buf[used++] = ch;
        if (ch == '\n')
            break;
    }

    buf[used] = '\0';
    return (ssize_t)used;
}

/*
 * 两轮登录业务保持为顺序代码。
 * username 和 password 都是普通局部变量, 由协程栈负责保存。
 */

```

```

static void handle_client(void)
{
    int id = coroutine_get_current_id();
    Coroutine *co = &scheduler.coroutines[id];
    int fd = co->client_fd;

    char username[64];
    char password[64];

    /*
     * TODO 2: 使用 co_read_line 完成顺序的两轮登录业务。
     *
     * 要求:
     * - 使用协程友好的按行读取函数读取用户名;
     * - 如果读取失败或对端关闭, 直接结束函数;
     * - 发送密码提示;
     * - 再读取密码;
     * - 读取成功后发送登录结果;
     * - 这个任务重点观察顺序代码结构, 不要求真正校验密码;
     * - 如果在本函数中主动 close(fd), 必须把 co->client_fd 设为 -1,
     *   避免 coroutine_wrapper 再次关闭同一个 fd。
     */
}

static int create_listen_socket(void)
{
    /*
     * TODO 3: 复用前置任务的实现, 创建非阻塞监听 socket。
     *
     * 要求:
     * - 创建 IPv4 TCP socket;
     * - 设置地址复用;
     * - 绑定 PORT 并开始监听;
     * - 设置为非阻塞;
     * - 成功时返回监听 fd, 失败时输出错误并结束程序。
     *
     * 可以复用任务二或任务三中的监听 socket 初始化顺序:
     * socket → setsockopt → bind → listen → set_nonblocking。
     */
}

int main(void)
{
    scheduler_init();

    int listen_fd = create_listen_socket();

    /*
     * TODO 4: 复用任务三的 epoll 初始化方法。
     *
     * 创建 epoll 实例, 把监听 fd 的可读事件注册进去,
     * 并将 LISTEN_TAG 保存为该事件的用户数据。
     */
}

```



```

* 要求:
* - 创建 epoll 实例并保存到全局 epoll_fd;
* - 监听 fd 关注可读事件;
* - 监听 fd 的事件数据必须能和普通协程事件区分;
* - 注册失败时打印错误并退出。
*/

/* 实验日志: 提示协程服务器已启动, 并给出手工测试命令。 */
printf("[Stage 4] 协程登录服务器启动, 监听 :%d\n", PORT);
printf("[Stage 4] 测试命令: nc 127.0.0.1 %d\n", PORT);

struct epoll_event events[MAX_EVENTS];

while (1) {
    int n;
    /*
    * TODO 5: 等待 epoll 事件, 无事件时一直阻塞。
    *
    * 要求: 没有事件时阻塞等待; 返回后只处理实际就绪的事件。
    */
    if (n < 0) {
        if (errno == EINTR)
            continue;
        perror("epoll_wait");
        break;
    }

    for (int i = 0; i < n; i++) {
        uint64_t event_data = events[i].data.u64;

        if (event_data == LISTEN_TAG) {
            for (;;) {
                int client_fd;
                /*
                * TODO 6: 复用任务三的非阻塞 accept 和错误处理。
                * 没有更多连接时结束本轮 accept。
                *
                * 要求:
                * - 从监听 fd 接收新连接;
                * - EAGAIN/EWOULDBLOCK 表示本轮 accept 完成, 应 break;
                * - EINTR/ECONNABORTED 可以继续;
                * - 其他错误打印 "accept" 并结束本轮 accept。
                */
                if (client_fd < 0) {
                    if (errno == EAGAIN ||
                        errno == EWOULDBLOCK)
                        break;
                    if (errno == EINTR ||
                        errno == ECONNABORTED)
                        continue;
                    perror("accept");
                    break;
                }
            }
        }
    }
}

```

```

        /*
        * TODO 7: 将 client_fd 设置为非阻塞,
        * 创建执行 handle_client 的协程, 并保存协程 ID。
        *
        * 要求:
        * - 新客户端 fd 必须设置为非阻塞;
        * - 创建一个执行 handle_client 的协程;
        * - 保存创建得到的协程 ID;
        * - co_id < 0 时说明没有空闲协程槽, 应关闭 client_fd。
        */
        int co_id;
        if (co_id < 0) {
            fprintf(stderr, "协程槽位已满\n");
            close(client_fd);
            continue;
        }

        /*
        * 首次运行会执行到第一个 co_read。
        * 若尚无数据, 协程注册 EPOLLIN 后主动 yield。
        */
        coroutine_resume(co_id);
    }
} else {
    int co_id = (int)event_data;
    coroutine_resume(co_id);
}
}

close(epoll_fd);
close(listen_fd);
return 0;
}

```

编译运行:

```
gcc -Wall -Wextra -O2 stage4_coroutine.c -o stage4
./stage4
```

另开终端:

```
nc 127.0.0.1 8080
```

先输入用户名并回车, 收到密码提示后再输入密码。

选择完成任务四时提交:

- 完整的 `stage4_coroutine.c`;
- 两轮登录运行结果;

- 与过渡部分 Reactor 状态机的代码结构和状态保存方式对比。

6. 预期结果与观察重点

阶段	线程数量	无数据时的行为	主要问题或改进
任务一	随连接数增长	每个线程阻塞在 <code>read</code>	线程资源耗尽
任务二	1	循环扫描并得到 <code>EAGAIN</code>	CPU 忙轮询
任务三	1	阻塞在 <code>epoll_wait</code>	只处理就绪连接
可选任务四	1 个内核线程、多协程	协程挂起，底层等待 <code>epoll</code>	I/O 等待机制不变，业务恢复顺序结构

任务二和任务三的 CPU 时间必须使用相同连接数和保持时间比较。

7. 实验思考题

基础必答题：

1. 任务一中 `connect()` 成功为什么不能证明服务端已经成功创建工作线程？客户端还必须检查什么？
2. `pthread_create()` 返回 `EAGAIN` 时，为什么操作系统不会在资源释放后自动替程序重新处理已经关闭的连接？
3. 任务二只有一个线程，为什么 4000 个客户端不发送数据时仍会消耗接近一个 CPU 核心？
4. 任务三的 `epoll_wait(..., -1)` 与任务二不断调用非阻塞 `read()` 在等待方式上有什么本质区别？
5. 如果使用 Reactor 实现两轮登录，为什么需要显式保存业务状态、用户名和收发进度？业务等待点继续增加时，代码结构会发生什么变化？

选择完成任务四时回答：

1. 为什么协程版本中的用户名和密码可以保留为普通局部变量，而 Reactor 版本通常需要把跨事件数据放入连接上下文？
2. 第二次调用 `co_read()` 后协程挂起。恢复时，CPU 为什么能回到第二次 `co_read()` 内部继续，而不是从 `handle_client()` 开头重新执行？

8. 自选进阶任务（二选一）

两个进阶任务都明显难于基础任务。选择其中一个独立完成，不提供框架代码。

8.1 挑战一：基于 `io_uring` 的 Proactor 两轮登录服务器

用 `io_uring` 重写两轮登录服务器，不得使用 `epoll`，也不得为每个连接创建一个工作线程。

要求：

1. 使用 SQE/CQE 完成 `accept`、`recv`、`send`；
2. 一个 `accept` 完成后立即提交下一个 `accept`；
3. 为每个连接维护用户名、密码、发送进度和会话状态；
4. 使用 `user_data` 将 CQE 映射回请求上下文；

5. 正确处理负数 CQE 结果、客户端关闭和部分发送；
6. 支持分段到达的用户名和密码，以换行符作为一轮输入结束；
7. 能同时保持大量尚未发送用户名的连接；
8. 在报告中解释 CQE 表示“I/O 已完成”，而 `epoll` 表示“fd 已就绪”。

环境：

```
sudo apt install liburing-dev
gcc -Wall -Wextra -O2 challenge_uring.c -o challenge_uring -luring
```

验收：

- 两个客户端交错输入时均能正确完成登录；
- 代码中不存在 `epoll_wait()`；
- 每个请求上下文均能在完成后正确释放；
- 部分发送不会截断提示或结果；
- 能说明运行环境不支持 `io_uring` 时的典型错误及排查方法。

8.2 挑战二：多 Reactor 多线程两轮登录服务器

将任务三的单线程 Reactor 扩展为“主线程 + 多个工作 Reactor”，并实现过渡部分描述的两轮登录协议。

要求：

1. 主线程只负责 `accept()`；
2. 至少创建 4 个工作线程，每个线程拥有独立的 `epoll` 实例；
3. 主线程使用 round-robin 分配连接；
4. 使用线程安全队列保存待接管 fd；
5. 使用 `eventfd` 唤醒工作线程；
6. 一个连接始终只归一个工作线程管理；
7. 每个工作线程实现完整两轮登录状态机；
8. 正确处理非阻塞读写、分段输入、部分发送和连接释放；
9. 输出每个工作线程接管的连接数量；
10. 分别使用 1、2、4 个工作线程测试并分析结果。

编译示例：

```
gcc -Wall -Wextra -O2 challenge_multireactor.c \
-o challenge_multireactor -lpthread
```

验收：

- 主线程不处理客户端读写；
- 不同工作线程拥有不同的 `epoll` fd；
- 跨线程传递使用队列和 `eventfd`；

- 不发生同一 fd 被多个线程同时管理；
- 两轮登录和大量等待连接均能正常工作；
- 能解释工作线程数量不是越多越好的原因。

9. 实验提交

基础部分提交：

1. 任务一至三的完整代码；
2. 任务一至三的最终运行结果；
3. 任务二、三 CPU 时间对比；
4. 五道基础实验思考题。

可选任务四提交：

1. 完整的 `stage4_coroutine.c`；
2. 两轮登录运行结果；
3. Reactor 状态机与协程代码结构的对比；
4. 两道任务四思考题。

进阶部分提交：

1. 选择的进阶任务；
2. 总体设计和关键数据结构；
3. 完整代码；
4. 功能测试和并发测试结果；
5. 错误处理与资源释放说明。

实验报告命名：

学号-姓名-操作系统实验七

提交地址：

<https://acm.scu.edu.cn/teach/>